

1. La bibliothèque numpy

L'utilisation de Python requiert l'importation préalable de bibliothèques (à chaque démarrage de l'environnement de travail). Nous allons en utiliser plusieurs mais, pour le moment, concentrons-nous sur la plus utile : la bibliothèque `numpy`. Celle-ci contient des commandes permettant de créer et manipuler des tableaux à plusieurs dimensions (des vecteurs en dimension 1, des matrices en dimension 2).

Pour le moment on limite notre étude aux vecteurs et on étudiera les matrices dans la partie VI.

- Pour importer la bibliothèque `numpy`, on utilise la commande

```
import numpy as np
```

- Si L est une liste de nombres, alors `np.array(L)` crée le vecteur ligne (tableau unidimensionnel) contenant, dans l'ordre, les nombres de la liste.
- On peut créer ainsi un vecteur « à la main » à partir d'une liste mais on peut aller plus vite pour des vecteurs remarquables (surtout si il y a beaucoup de coordonnées) : soient $n \in \mathbb{N}$ et $(a, b, h) \in \mathbb{R}^3$ implémentés en Python par n, a, b et h respectivement :

- `np.zeros(n)` renvoie un vecteur avec n coordonnées toutes nulles.

- `np.ones(n)` renvoie un vecteur avec n coordonnées valant toutes 1.

- `np.linspace(a,b,n)` renvoie un vecteur contenant n valeurs comprises entre a et b (a et b inclus) régulièrement espacées.

- Si $a < b$ et $h > 0$, `np.arange(a,b,h)` renvoie un vecteur contenant $a, a + h, a + 2h, a + 3h, a + 4h$ etc. jusqu'à b (b non inclus). Si $a > b$ et $h < 0$, alors cela fait la même chose mais dans l'ordre décroissant.

Par exemple `np.arange(3,4,0.2)` renvoie

`array([3., 3.2, 3.4, 3.6, 3.8])`

et `np.arange(6,0,-1)` renvoie `array([6, 5, 4, 3, 2, 1])`

- On accède aux coordonnées du vecteur (et on les modifie) de la même façon qu'on accède aux éléments d'une liste.
- Si on ne connaît pas le nombre de coordonnées d'un vecteur V , il suffit d'utiliser la commande `len(V)`.
- On peut effectuer des opérations coefficient par coefficient sur des vecteurs via les commandes `+`, `-`, `*`, `/`, `**`.

Par exemple :

- `V**3` renvoie le vecteur V dont tous les coordonnées ont été élevées à la puissance 3.

- Si `V=np.array([1,2,3])` et `W=np.array([0,7,-1])`, alors `V*W` renvoie `array([0,14,-3])`.



On ne peut sommer, soustraire, multiplier, diviser deux vecteurs entre eux que s'ils ont le même nombre de coordonnées.

- On peut comparer deux vecteurs coefficient par coefficient ou comparer un vecteur coefficient par coefficient avec un nombre en utilisant les commandes `==`, `>`, `<`, `>=`, `<=`, `!=`. Le résultat est une matrice de booléens.

Par exemple, si `V=np.array([-1,2,3,4])` et `W=np.array([7,1,3,5])`, alors `V>=W` renvoie `array([False,True,True,False])`.

2. Fonctions usuelles

Les fonctions $\exp$, $\ln$, $\sin$, $\cos$, racine carrée, valeur absolue et partie entière sont implémentées en Python respectivement par `np.exp`, `np.log`, `np.sin`, `np.cos`, `np.sqrt`, `np.abs` et `np.floor`.



`log` désigne le logarithme népérien pour Python (c'est la convention anglo-saxonne).

Ces fonctions peuvent s'appliquer à des variables numériques ou vectoriellement (à des vecteurs réels) coordonnée par coordonnée.

Par exemple `np.log(np.e)` renvoie 1.0.

La commande `np.sin([0,np.pi,np.pi/2])` renvoie `array([0.0,0.0,1.0])`.

Remarques :

- Si $x \in D_{\tan}$ est implémenté en Python par `x`, alors `np.sin(x)/np.cos(x)` renvoie (une approximation de) $\tan(x)$.
- Si $x \in \mathbb{R}_+$ et $n \in \mathbb{N} \setminus \{0; 1\}$ sont implémentés en Python par `x` et `n`, alors `x**(1/n)` renvoie (une approximation de) $\sqrt[n]{x}$.
- Si $x \in \mathbb{R}$ est implémenté en Python par `x`, alors `np.arctan(x)` renvoie (une approximation de) $\text{Arctan}(x)$ mais cette commande n'est pas exigible. On peut aussi programmer la fonction Arctan avec la méthode des rectangles (cf. partie VII) ou un algorithme de dichotomie (cf. partie VI).